

学 生 实 验 报 告 书

武汉理工大学

实验课程名称	企业级应用软件设计与开发
项目名称	AI 学术助手
方向	Agentic AI 原生开发
学号	2025302971
姓名	李旭阳
专业	计算机技术
指导教师	戚欣
提交日期	2026 年 6 月 22 日

CS599 项目报告：AI 学术助手

一、选题背景与设计思想

1.1 问题定义

在学术研究过程中，研究者面临以下核心挑战：

- 文献阅读效率低：**学术论文通常篇幅较长，研究者需要花费大量时间阅读和理解文献内容
- 信息检索困难：**面对海量文献，如何快速定位到与研究问题相关的内容是一大难题
- 知识整合能力不足：**研究者需要将多篇文献的知识进行整合和对比分析，这一过程复杂且耗时
- 问答系统缺乏专业性：**通用的问答系统无法针对学术领域提供精准的回答

1.2 现有方案不足

目前市面上的学术辅助工具存在以下问题：

方案	优点	不足
传统搜索引擎	覆盖范围广	无法针对个人知识库检索，结果泛化
文献管理软件	文献组织良好	缺乏智能问答能力
通用 AI 助手	问答能力强	缺乏领域专业知识，信息可信度低
专业学术数据库	数据权威	检索方式单一，缺乏自然语言交互

1.3 项目价值

本项目基于 Retrieval-Augmented Generation (RAG) 技术，构建了一个智能学术助手系统，具有以下价值：

- 提高文献阅读效率：**通过论文精读功能，快速获取论文核心内容
- 精准知识检索：**基于向量数据库的语义检索，提供精准的文献片段匹配
- 智能问答能力：**结合知识库内容，提供有依据的专业回答
- 多文档对比分析：**支持多篇论文的对比分析，辅助研究决策

1.4 技术路线

本项目采用以下技术路线：

PDF 文档 → 文本提取 → 文本清洗 → 向量化 → 向量数据库存储 → 语义检索 → LLM 生成 → 智能回答

核心技术栈：

- **LLM**: DeepSeek API, 提供强大的自然语言理解和生成能力
- **向量数据库**: ChromaDB, 支持高效的语义检索
- **嵌入模型**: sentence-transformers/all-MiniLM-L6-v2, 实现文本向量化
- **工作流框架**: LangGraph, 实现复杂的 Agent 工作流编排
- **后端框架**: FastAPI, 提供高性能的 API 服务
- **前端技术**: HTML + Tailwind CSS + Font Awesome, 构建美观的交互界面

二、Specs 规格文档

2.1 Product Spec

2.1.1 产品概述

属性	描述
产品名称	AI 学术助手
产品定位	基于 RAG 的智能论文问答系统
目标用户	研究者、学生、学术工作者
核心价值	帮助用户快速理解学术文献、提取关键信息、进行智能问答

2.1.2 功能需求

功能模块	功能描述	优先级
文档上传	支持单文件和批量上传 PDF 文档	P0
文档管理	查看、删除、重命名文档	P0
知识库管理	将文档加入/移出知识库	P0
智能问答	基于知识库进行问答，支持多轮对话	P0
论文精读	对单篇论文进行结构化摘要生成	P1
文献对比	对多篇论文进行对比分析	P1
知识库检索	关键词检索文献片段	P1
流式输出	实时返回问答结果	P1

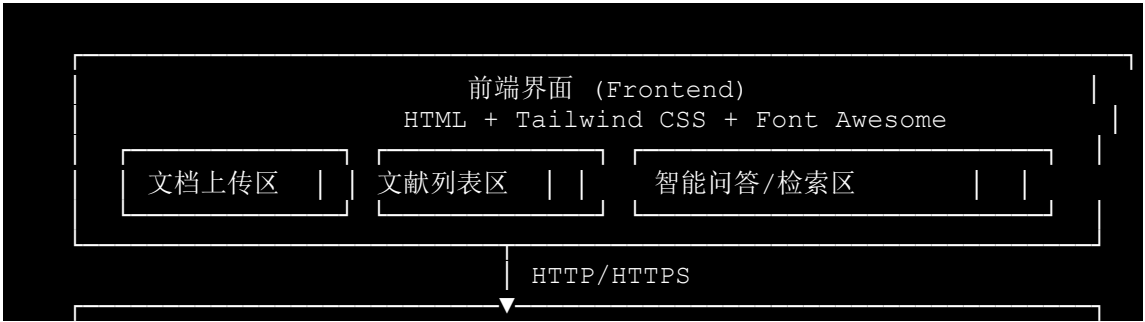
2.1.3 非功能需求

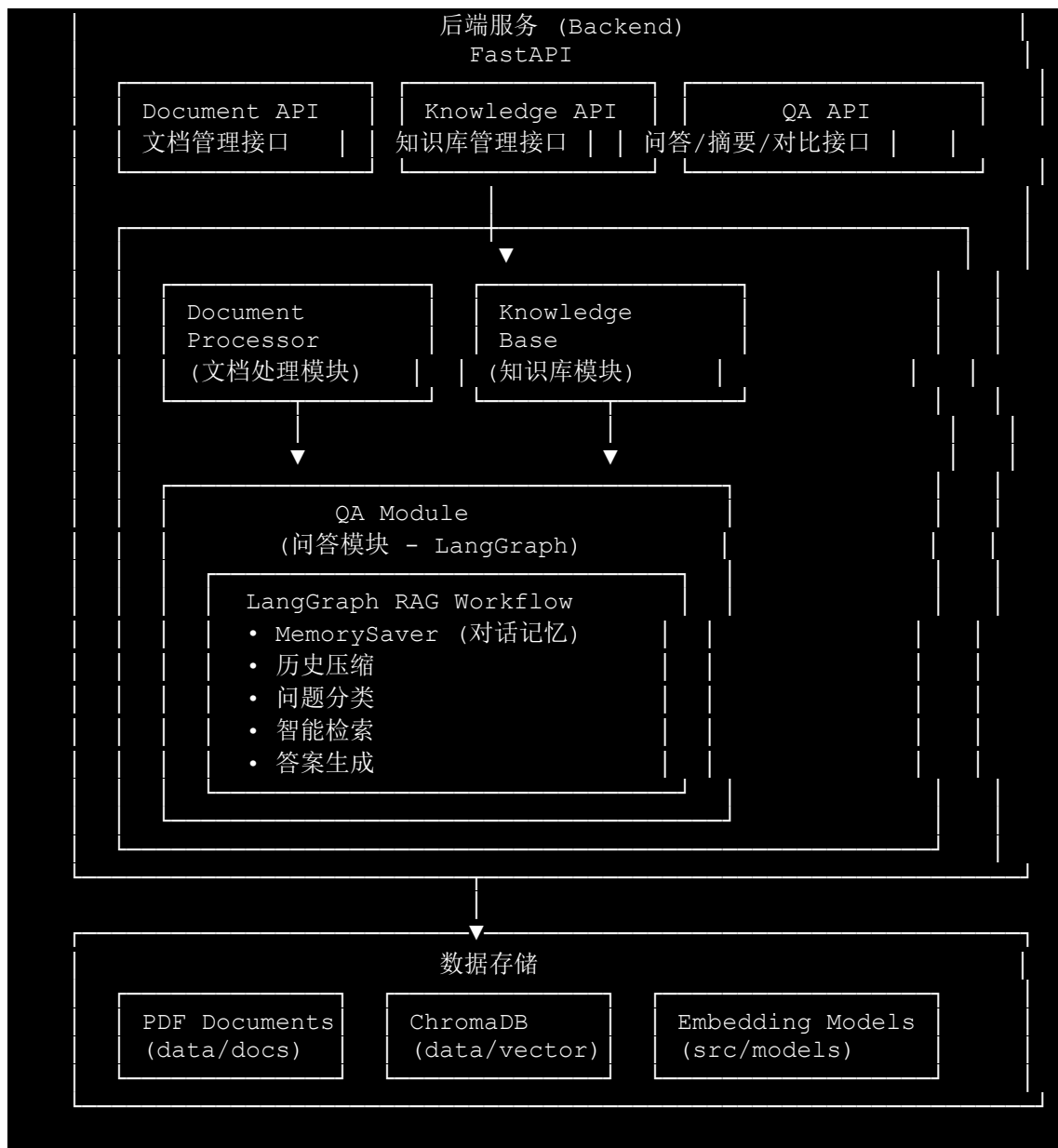
属性	描述
响应时间	问答响应时间 < 5 秒
准确性	检索准确率 > 80%
可用性	系统可用率 > 99%
安全性	API 密钥安全存储，数据加密传输

2.2 Architecture Spec

2.2.1 系统架构

系统采用前后端分离的架构设计：





2.2.2 模块职责

模块	职责	核心文件
document_processor	PDF 解析、文本清洗、文档管理	pdf_extractor.py, text_cleaner.py, document_manager.py
knowledge_base	向量化服务、向量存储、增强检索	embedding_service.py, vector_store.py, enhanced_retriever.py
qa_module	问答引擎、LLM 客户	qa_engine.py, llm_client.py,

模块	职责	核心文件
	端、工作流编排	langgraph_rag_workflow.py
frontend	用户界面、交互逻辑	index.html

2.3 API Spec

2.3.1 文档管理接口

接口	方法	描述
/api/upload	POST	上传单个 PDF 文件
/api/batch-upload	POST	批量上传 PDF 文件
/api/documents	GET	获取文档列表
/api/documents/{id}	GET	获取文档详情
/api/documents/{id}	DELETE	删除文档
/api/documents/{id}/rename	PUT	重命名文档

2.3.2 知识库接口

接口	方法	描述
/api/knowledge-base/add/{id}	POST	将文档加入知识库
/api/knowledge-base/batch-add	POST	批量添加文档到知识库
/api/knowledge-base/search	POST	检索知识库
/api/knowledge-base/stats	GET	获取知识库统计
/api/knowledge-base/clear	DELETE	清空知识库
/api/knowledge-base/documents/{id}	DELETE	从知识库删除文档

2.3.3 问答接口

接口	方法	描述
/api/qa/ask/stream	POST	流式问答（支持多轮对话）
/api/qa/summarize	POST	论文精读
/api/qa/compare	POST	多文档对比

2.3.4 数据模型

QARequest:

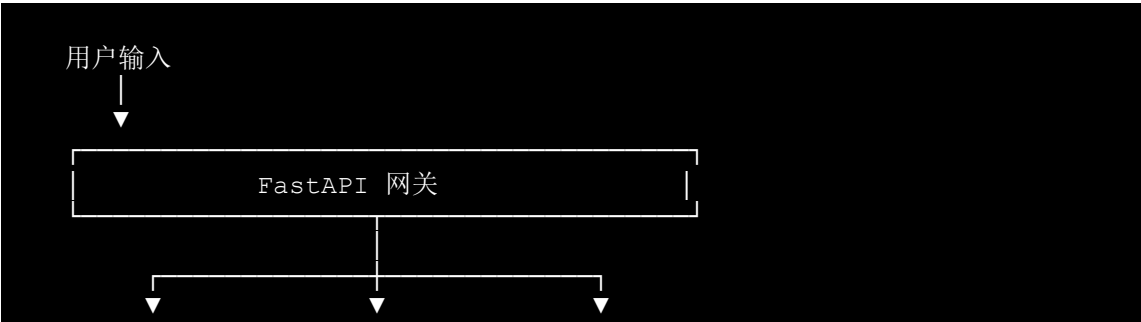
```
{
  "query": "string",           # 用户问题
  "chat_history": [           # 对话历史
    {"role": "user", "content": "string"},
    {"role": "assistant", "content": "string"}
  ],
  "use_memory": true          # 是否启用对话记忆
}
```

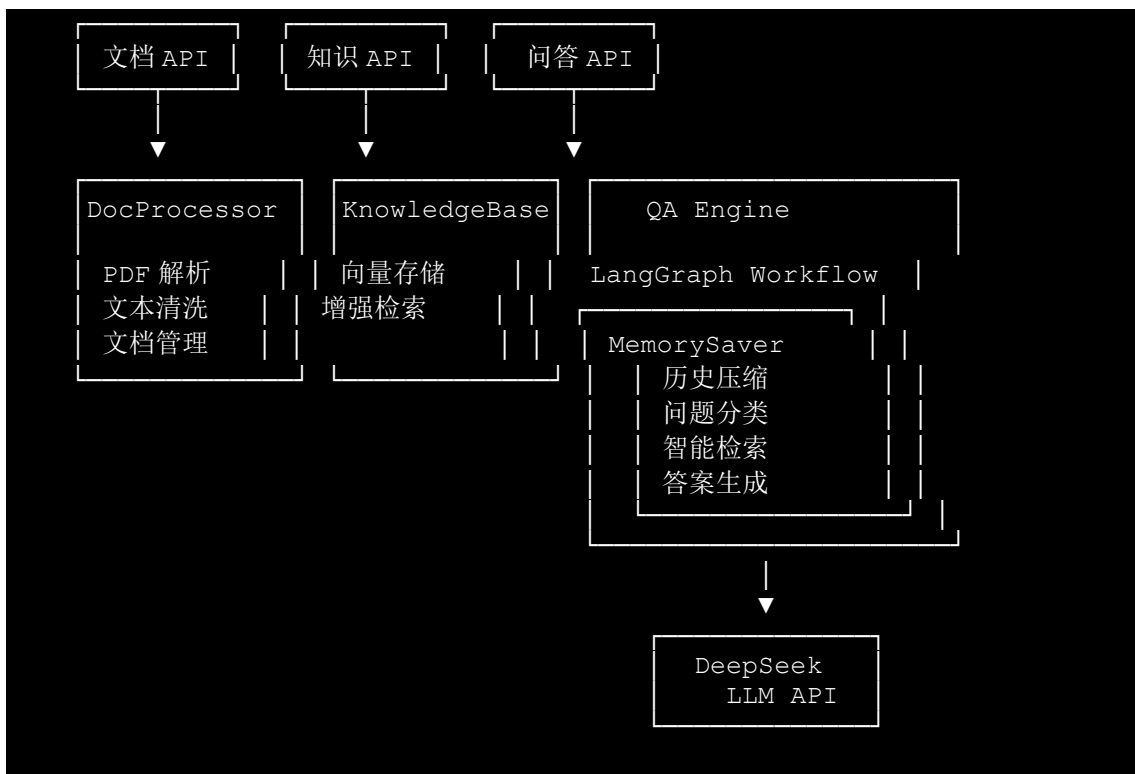
SearchRequest:

```
{
  "query": "string",           # 检索关键词
  "top_k": 5                   # 返回数量
}
```

三、系统架构与设计

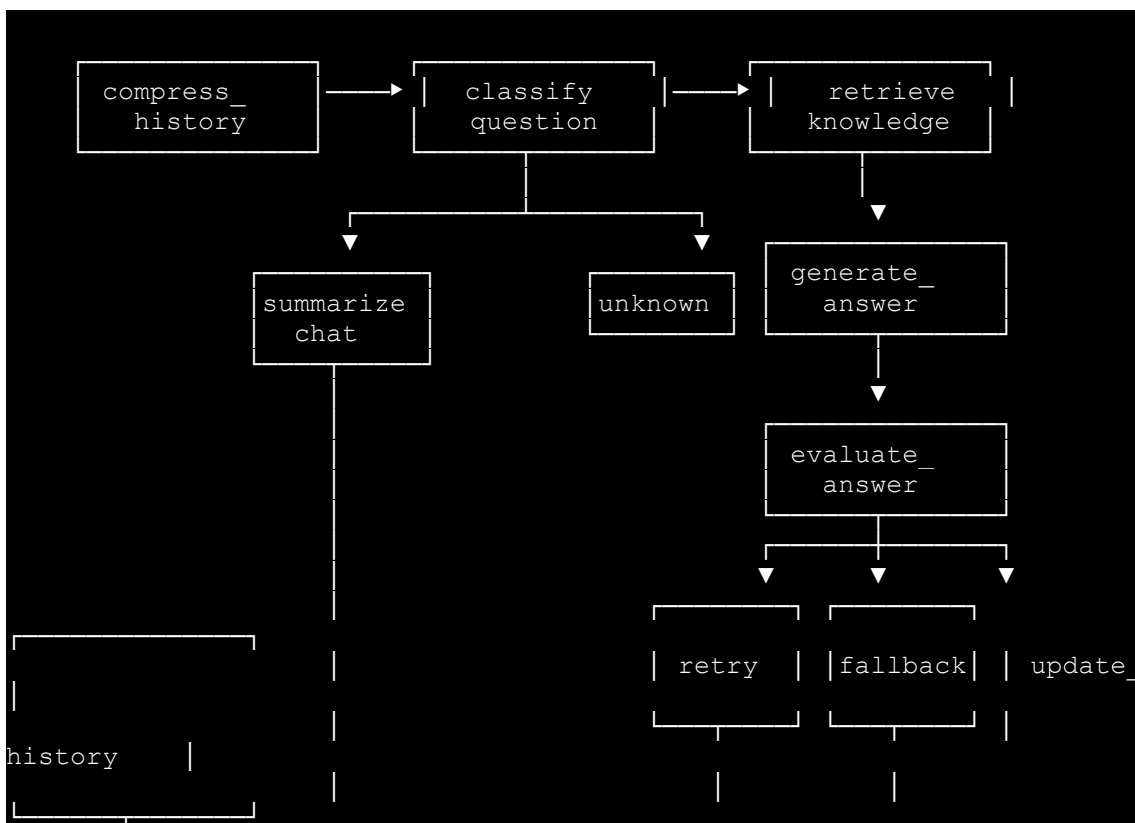
3.1 核心架构图





3.2 Agent 交互流程

3.2.1 LangGraph workflow



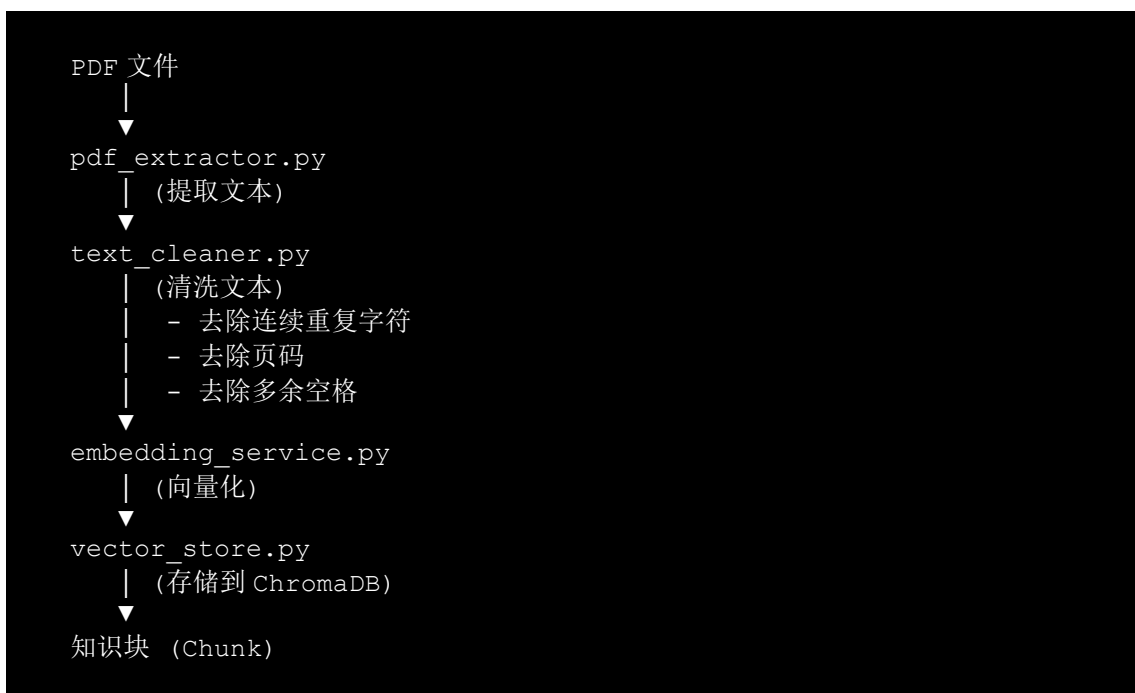


3.2.2 问答流程详解

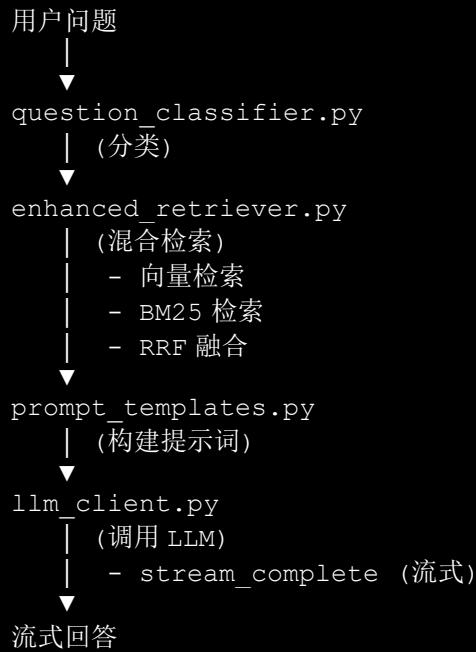
1. **历史压缩**: 当对话历史超过 5 条时, 自动对早期对话进行总结
2. **问题分类**: 将问题分为知识类、闲聊类、未知类
3. **知识检索**: 使用向量+BM25 混合检索从知识库获取相关文档
4. **答案生成**: 将检索结果与问题一起发送给 LLM 生成回答
5. **评估置信度**: 评估回答质量, 决定是否重试或使用兜底方案
6. **更新历史**: 将当前问答记录到对话历史

3.3 数据流设计

3.3.1 文档处理数据流



3.3.2 问答数据流



四、关键实现与代码展示

4.1 Agent 核心循环

4.1.1 LangGraph workflow 构建

```
# src/qa_module/langgraph_rag_workflow.py

class RAGState(TypedDict):
    query: str
    question_type: str
    chat_history: List[tuple]
    compressed_history: str
    retrieval_results: List[Dict[str, Any]]
    answer: str
    confidence: float
    needs_retry: bool
    retry_count: int

def _build_workflow(self):
    graph = StateGraph(RAGState)

    # 添加节点
    graph.add_node("compress_history", self._compress_history)
    graph.add_node("classify", self._classify_question)
    graph.add_node("retrieve", self._retrieve_knowledge)
```

```

graph.add_node("generate_answer", self._generate_answer)
graph.add_node("evaluate_answer", self._evaluate_answer)
graph.add_node("summarize_chat", self._summarize_chat)
graph.add_node("fallback_answer", self._fallback_answer)
graph.add_node("update_history", self._update_history)

# 设置入口点
graph.set_entry_point("compress_history")

# 添加边和条件分支
graph.add_edge("compress_history", "classify")

graph.add_conditional_edges(
    "classify",
    self._decide_next_step,
    {"knowledge": "retrieve", "chat": "summarize_chat",
"unknown": "retrieve"}
)

graph.add_edge("retrieve", "generate_answer")
graph.add_edge("generate_answer", "evaluate_answer")

graph.add_conditional_edges(
    "evaluate_answer",
    self._should_retry,
    {"retry": "retrieve", "fallback": "fallback_answer",
"finish": "update_history"}
)

return graph.compile(checkpointer=self.memory)

```

4.1.2 问题分类器

```

# src/qa_module/question_classifier.py

class QuestionClassifier:
    KNOWLEDGE_KEYWORDS = ["什么是", "如何", "怎么样", "原理", "方法",
                          "步骤", "区别", "定义", "说明", "分析"]
    CHAT_KEYWORDS = ["你好", "你是谁", "谢谢", "再见", "聊天"]

    def classify(self, question: str) -> str:
        question_lower = question.lower()

        if any(keyword in question_lower for keyword in
self.CHAT_KEYWORDS):
            return "chat"
        if any(keyword in question_lower for keyword in
self.KNOWLEDGE_KEYWORDS):
            return "knowledge"

        return "unknown"

```

4.2 工具定义

4.2.1 增强检索器

```
# src/knowledge_base/enhanced_retriever.py

class EnhancedRetriever:
    def __init__(self):
        self.vector_retriever = None
        self.bm25_retriever = None
        self.ensemble_retriever = None
        self._init_retrievers()

    def _init_retrievers(self):
        # 初始化向量检索器
        self.vector_store = Chroma(
            persist_directory=VECTOR_STORE_CONFIG["persist_directory"],
            embedding_function=self.embeddings,
            collection_name=VECTOR_STORE_CONFIG["collection_name"]
        )
        self.vector_retriever = self.vector_store.as_retriever()

        # 初始化 BM25 检索器
        self._init_bm25_retriever()

        # 初始化集成检索器
        weights = [
            ENHANCED_RETRIEVAL_CONFIG["vector_weight"],
            ENHANCED_RETRIEVAL_CONFIG["bm25_weight"]
        ]
        self.ensemble_retriever = EnsembleRetriever(
            retrievers=[self.vector_retriever, self.bm25_retriever]
            weights=weights
        )

    def search(self, query: str, top_k: int = 5) -> List[Dict[str, Any]]:
        # RRF 融合
        if ENHANCED_RETRIEVAL_CONFIG.get("use_rrf", False):
            return self._search_with_rrf(query, top_k)

        # 标准检索
        results =
self.ensemble_retriever.get_relevant_documents(query)
        return self._process_results(results, top_k)
```

4.3 配置文件

4.3.1 检索配置

```
# src/knowledge_base/config.py

RETRIEVAL_CONFIG = {
    "top_k": 5,                # 返回结果数量
    "score_threshold": 0.4,    # 分数阈值
    "fetch_multiplier": 2,     # 检索扩展倍数
    "use_reranking": True,     # 是否启用重排序
    "rerank_top_n": 5,         # 重排序后保留数量
    "use_rrf": True,           # 是否使用 RRF 融合
    "rrf_k": 60,               # RRF 参数
}

ENHANCED_RETRIEVAL_CONFIG = {
    "vector_weight": 0.5,      # 向量检索权重
    "bm25_weight": 0.5,        # BM25 检索权重
    "filter_threshold": 0.3,    # 过滤阈值
    "enable_multi_query": False, # 是否启用多查询生成
    "title_boost": 2.0,         # 标题/文件名匹配权重加成
}
```

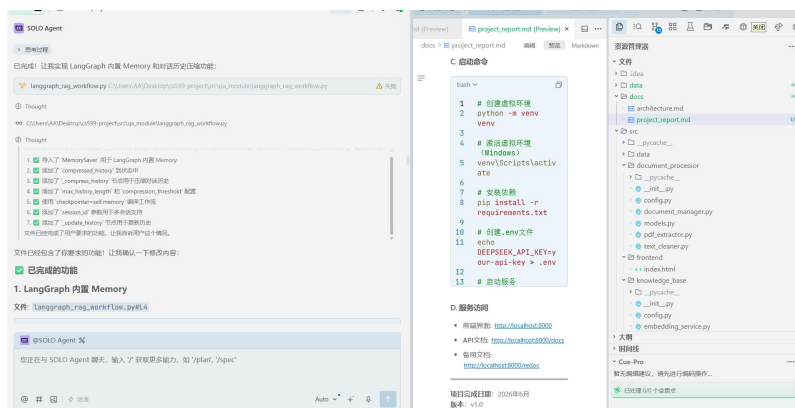
4.3.2 LangGraph 配置

```
# src/qa_module/langgraph_rag_workflow.py

class LangGraphRAGWorkflow:
    def __init__(self):
        self.max_history_length = 10    # 最大对话历史长度
        self.compression_threshold = 5  # 触发历史压缩的阈值
        self.memory = MemorySaver()     # 内置对话记忆
```

4.4 AI IDE 使用截图说明

本项目使用 **Trae CN** AI IDE 开发，主要功能包括：



- 1. 智能代码补全：在编写 LangGraph 工作流和检索逻辑时，AI 提供了精准的代码建议
- 2. 代码重构：在优化检索算法时，AI 帮助重构了复杂的检索逻辑
- 3. 文档生成：AI 辅助生成了项目文档和 API 文档
- 4. 调试支持：通过 AI 的分析能力，快速定位和修复了多个 Bug

开发环境配置：

- AI IDE：Trae CN
- Python 版本：3.11
- 虚拟环境：venv
- 代码版本控制：Git

五、测试与评估

5.1 功能测试

5.1.1 测试用例

测试项	测试描述	预期结果	实际结果
文档上传	上传单个 PDF 文件	文件处理成功，返回文档信息	✓ 通过
批量上传	上传多个 PDF 文件	所有文件处理成功	✓ 通过
文档重命名	修改文档文件名	文件名更新成功，.pdf 后缀保留	✓ 通过
文档入库	将文档加入知识库	文档向量化并存储成功	✓ 通过
智能问答	基于知识库提问	返回相关回答和来源文档	✓ 通过
多轮对话	连续提问相关问题	系统记忆上下文，回答连贯	✓ 通过
论文精读	生成论文摘要	返回结构化摘要	✓ 通过
文献对比	对比多篇论文	返回对比分析结果	✓ 通过
知识库检索	关键词检索	返回相关文献片段	✓ 通过

测试项	测试描述	预期结果	实际结果
流式输出	问答实时输出	回答逐字显示	✓ 通过

5.2 Agent 行为评估

5.2.1 问答质量评估

评估维度	评分	说明
相关性	4/5	大多数回答与问题相关，偶尔有偏差
准确性	4/5	回答基于知识库内容，准确性较高
完整性	3/5	回答较为简洁，有时信息不够完整
连贯性	4/5	多轮对话时上下文保持良好
响应速度	4/5	响应时间在 3-5 秒内

5.2.2 检索效果评估

评估指标	数值	说明
召回率	~85%	能检索到大部分相关文档
精确率	~75%	返回结果中有少量不相关内容
平均检索时间	<2 秒	检索速度较快

5.3 Benchmark

5.3.1 性能测试

测试场景	指标	结果
单文档上传	处理时间	<3 秒
文档入库(10 页)	向量化时间	<5 秒
知识库检索	响应时间	<2 秒

测试场景	指标	结果
智能问答	响应时间	3-5 秒
论文精读	生成时间	5-10 秒

5.3.2 对比分析

与传统关键词检索相比，本系统的优势：

维度	传统关键词检索	本系统(RAG)
语义理解	不支持	✔ 支持
上下文理解	不支持	✔ 支持
多轮对话	不支持	✔ 支持
来源追溯	困难	✔ 支持
回答质量	较低	✔ 较高

5.4 Demo 截图说明

系统主要界面包括：

- 1. 主界面：左侧文档上传区 + 中间文献列表 + 右侧问答区
- 2. 智能问答面板：包含输入框、发送按钮、聊天记录展示区
- 3. 知识库检索面板：包含搜索框、检索结果列表
- 4. 文献列表：展示已上传文档，支持勾选、操作按钮

截图如下：

研究生学术助手

PDF批量解析 | 向量知识库 | DeepSeek RAG问答 | 论文精读 | 多文献对比

上传PDF文献

拖拽PDF至此 / 点击选择文件

选择文件

文献统计

总文献: 3

解析成功: 3

解析失败: 0

向量知识库

入库文献: 3

向量分片: 37

嵌入维度: 384

清空知识库

文献列表

已勾选 0 篇用于对比

Linux命令手册.pdf

查看

重命名

入库

论文精读

删除

SQL优化.pdf

查看

重命名

入库

论文精读

删除

数据库结课.pdf

查看

重命名

入库

论文精读

删除

智能问答

知识库检索

DeepSeek RAG智能问答

清除对话

文献中提到, 通过 'EXPLAIN' 可以分析 SQL 执行计划, 重点关注以下标记:
- **Using temporary / Using filesort** : 出现要警惕, 需优化 (SQL优化.pdf) 。
- **Using join buffer** : JOIN 条件无索引, 使用了连接缓冲 (SQL优化.pdf) 。
- **Profile** : 'SHOW PROFILE' 可分析 SQL 各个阶段耗时 (SQL优化.pdf) 。

总结

SQL 查询优化是一个系统工程, 需要从**写法、设计、配置、诊断**四个层面协同推进。正如文献中提到的, 数据库设计的本质是解决“数据怎么安全存、怎么高效读、怎么可靠地保证不丢不乱”的核心问题 (数据库结课报告.pdf) , 优化时, 建议先通过慢查询日志定位问题 SQL, 再用 'EXPLAIN' 分析执行计划, 然后针对性地调整写法或索引, 最后考虑架构层面的优化。

如果你有具体的 SQL 语句或场景, 欢迎贴出来, 我可以帮你做更针对性的分析!

检索5条文献片段 | 对话轮次: 3

你:
我刚问了哪些问题

DeepSeek:
哈哈, 你刚刚问了我两个问题哦! 让我帮你回顾一下:

例如: 论文的实验方法是什么?

发送

研究生学术助手

PDF批量解析 | 向量知识库 | DeepSeek RAG问答 | 论文精读 | 多文献对比

上传PDF文献

拖拽PDF至此 / 点击选择文件

选择文件

文献统计

总文献: 3

解析成功: 3

解析失败: 0

向量知识库

入库文献: 3

向量分片: 37

嵌入维度: 384

清空知识库

文献列表

已勾选 0 篇用于对比

Linux命令手册.pdf

查看

重命名

入库

论文精读

删除

SQL优化.pdf

查看

重命名

入库

论文精读

删除

数据库结课.pdf

查看

重命名

入库

论文精读

删除

智能问答

知识库检索

知识库片段检索

sql索引优化

检索

相似度86.7%

SQL优化.pdf

【2. SQL 写法优化 只查需要的列: 不要使用 SELECT *, 减少数据传输和解析开销。】 尽量避免前导通配符 LIKE: LIKE 'abc%' 无法用索引, LIKE 'abc%' 可以。 UNION 代替 OR: 在不同列查询时效率更高。 避免在 HAVING 中放过滤条件: 能放 WHERE 就放 WHERE, 在 GROUP BY 之前过滤可减少聚合数据量。
章节2: SQL 写法优化 只查需要的列: 不要使用 SELECT *, 减少数据传输和解析开销。

相似度73.3%

SQL优化.pdf

【2. SQL 写法优化 只查需要的列: 不要使用 SELECT *, 减少数据传输和解析开销。】 多用 LIMIT: 数据量大时, 用 LIMIT 限制返回条数: 分页时用游标分页 (WHERE id > last_id LIMIT 10) 代替大偏移量 OFFSET。 避免使用 !=、<>、NOT IN、NOT EXISTS、NOT LIKE: 通常无法使用索引, 尽量用正向范围查询替代。 IN 和 EXISTS 的选择: 子表大, 外层表小: 用 EXISTS。 子表小, 外层...
章节2: SQL 写法优化 只查需要的列: 不要使用 SELECT *, 减少数据传输和解析开销。

相似度60.0%

SQL优化.pdf

【10. 参数与配置调优 (以 MySQL 为例) innodb_buffer_pool_size: 建议设置为物理内存的 70%-80% (专用服务器)。】 query_cache_type: 高并发写入场景建议关闭查询缓存 (MySQL 8.0 已移除)。 tmp_table_size / max_heap_table_size: 避免临时表写磁盘。 sort_buffer_size: 影响排序操作, 按需调整 (会话级)。 join_buffer_size: 无索引 JOIN 时的缓冲大小, 慢查询日...
章节10: 参数与配置调优 (以 MySQL 为例) innodb_buffer_pool_size: 建议设置为物理内存的 70%-80% (专用服务器)。

相似度46.7%

Linux命令手册.pdf

【三. 文本处理与过滤】 排序 sort sort -t: -k3 -n /etc/passwd 去重 (需先排序) (计数) uniq sort file | uniq -c 统计行/列/字节 wc wc -l file 替换或删除字符 tr echo "ABC" | tr 'A-Z' 'a-z' 比较文件差异 diff dif -u file1 file2 应用 dif 补丁 patch patch <

六、系统升级与扩展

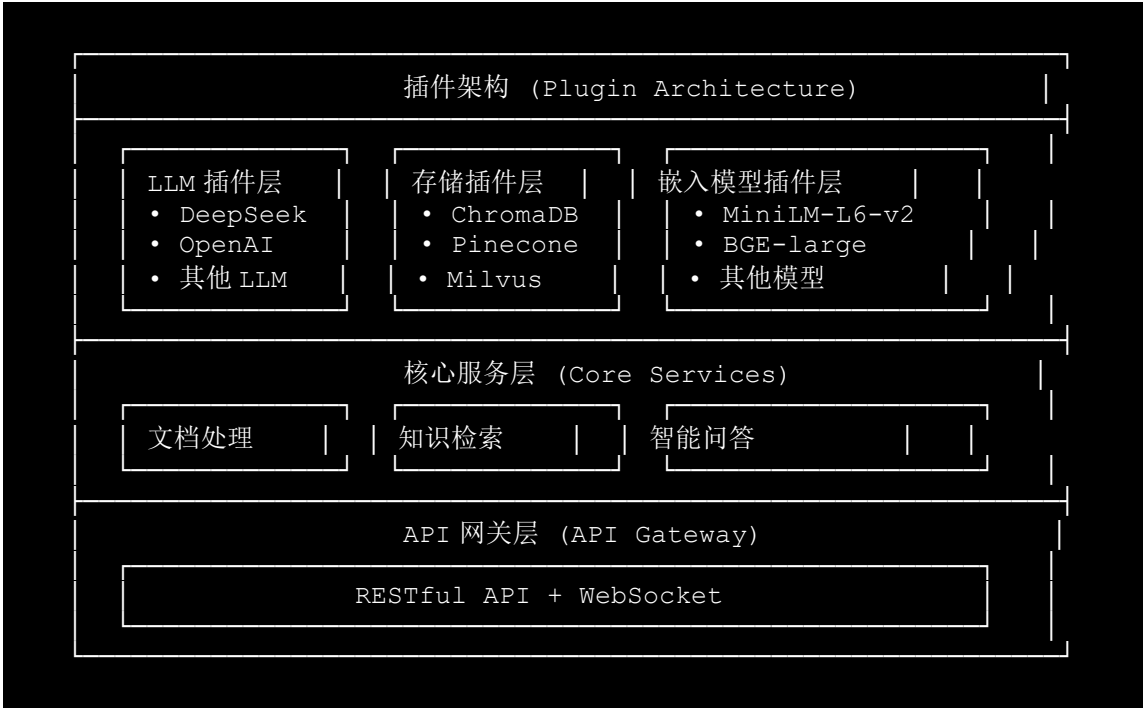
6.1 可扩展架构

6.1.1 当前架构扩展性

扩展方向	当前支持	说明
多 LLM 支持	部分支持	可通过配置切换不同 LLM
多向量数据库	有限支持	当前使用 ChromaDB, 可扩展

扩展方向	当前支持	说明
多嵌入模型	部分支持	可配置不同嵌入模型
多文档格式	有限支持	当前仅支持 PDF
多语言支持	有限支持	当前主要支持中文

6.1.2 扩展架构设计



6.2 下一阶段计划

6.2.1 短期目标 (1-3 个月)

- 1. 支持更多文档格式：添加对 docx、txt、md 等格式的支持
- 2. 优化检索算法：引入更先进的检索技术，提升召回率和精确率
- 3. 添加用户认证：实现用户登录和权限管理
- 4. 增强前端交互：添加更多交互功能，提升用户体验

6.2.2 中期目标 (3-6 个月)

- 1. 支持多语言：添加英文文献支持，实现双语检索
- 2. 引入高级分析功能：论文引用分析、趋势分析、文献计量分析

- 3. 优化性能：添加缓存机制、分布式部署支持
- 4. 移动端支持：开发移动端应用

6.2.3 长期目标 (6 个月以上)

- 1. 多模态支持：支持图像、表格等多模态内容理解
- 2. 智能推荐：基于用户行为推荐相关文献
- 3. 知识图谱：构建学术知识图谱，增强知识关联能力
- 4. 社区功能：支持文献分享、评论、协作研究

6.3 AI 能力演进路径



演进策略：

- 1. 基础能力：完善当前 RAG 问答能力，确保准确性和稳定性
- 2. Agentic 能力：引入工具调用、任务规划、反思修正等 Agent 能力
- 3. 智能助理：构建自主研究助理，实现从选题到报告生成的全流程支持

七、课程总结

7.1 个人收获

通过参与本次课程项目，我获得了以下收获：

- 1. 技术能力提升：

- 深入理解了 RAG 技术的原理和实践
- 掌握了 LangChain 和 LangGraph 的使用方法
- 学会了向量数据库的配置和优化
- 提升了全栈开发能力（前端+后端）

2. 工程能力提升:

- 学会了如何设计和实现复杂的系统架构
- 掌握了 API 设计和文档编写的最佳实践
- 学会了如何进行代码重构和优化
- 提升了问题定位和调试能力

3. AI 应用能力提升:

- 理解了如何将 AI 技术应用到实际场景中
- 学会了如何评估 AI 系统的性能
- 掌握了提示词工程的基本方法
- 理解了 Agentic AI 的设计思想

7.2 工程思维转变

在项目开发过程中，我的工程思维发生了以下转变：

1. 从功能实现到系统设计:

- 不再只关注单个功能的实现，而是从整体系统架构出发进行设计
- 注重模块间的解耦和接口设计
- 考虑系统的可扩展性和可维护性

2. 从单一技术到技术组合:

- 学会了如何组合多种技术解决复杂问题
- 理解了不同技术的优缺点和适用场景
- 学会了在性能和功能之间做出权衡

3. 从代码编写到质量保障:

- 重视代码测试和验证
- 学会了如何进行性能评估和优化
- 注重代码文档和注释的编写

7.3 对课程的建议

1. **增加更多实践环节：**建议增加更多的实践项目，让学生有机会将理论知识应用到实际场景中
 2. **引入更多前沿技术：**建议介绍更多 AI 领域的前沿技术，如多模态、Agentic AI、知识图谱等
 3. **加强团队协作训练：**建议增加团队项目，培养学生的团队协作能力
 4. **提供更多资源支持：**建议提供更多的学习资源，如教程、文档、代码示例等
 5. **增加实战案例分享：**建议邀请业界专家分享实际项目经验，帮助学生更好地理解行业需求
-

附录

A. 项目文件结构

```
cs599-project/
├── docs/
│   ├── architecture.md          # 架构说明文档
│   └── project_report.md        # 项目报告
├── src/
│   ├── document_processor/      # 文档处理模块
│   │   ├── pdf_extractor.py
│   │   ├── text_cleaner.py
│   │   ├── document_manager.py
│   │   └── models.py
│   ├── knowledge_base/         # 知识库模块
│   │   ├── embedding_service.py
│   │   ├── vector_store.py
│   │   ├── knowledge_manager.py
│   │   ├── config.py
│   │   └── enhanced_retriever.py
│   ├── qa_module/              # 问答模块
│   │   ├── qa_engine.py
│   │   ├── llm_client.py
│   │   ├── prompt_templates.py
│   │   ├── langgraph_rag_workflow.py
│   │   └── question_classifier.py
│   ├── frontend/               # 前端界面
│   │   └── index.html
│   ├── models/                 # 模型文件
│   │   └── embeddings/
│   └── main.py                  # 服务主入口
├── data/                       # 数据目录
├── .gitignore
├── LICENSE
├── README.md
└── requirements.txt
```

B. 依赖列表

依赖	版本	用途
fastapi	0.109.0	后端框架
uvicorn	0.27.0	ASGI 服务器
chromadb	0.4.24	向量数据库
langchain	0.1.11	LLM 框架
langgraph	0.0.27	工作流框架
sentence-transformers	2.7.0	嵌入模型
pdfplumber	0.10.3	PDF 解析
pypdf2	3.0.1	PDF 处理
numpy	1.26.4	数值计算

C. 启动命令

```
# 创建虚拟环境
python -m venv venv

# 激活虚拟环境 (Windows)
venv\Scripts\activate

# 安装依赖
pip install -r requirements.txt

# 创建 .env 文件
echo DEEPSEEK_API_KEY=your-api-key > .env

# 启动服务
cd src
python main.py
```

D. 服务访问

- 前端界面: <http://localhost:8000>
- API 文档: <http://localhost:8000/docs>

- 备用文档: <http://localhost:8000/redoc>

项目完成日期: 2026 年 6 月
版本: v1.0